

Introdução ao Python e Web Scraping

Gabriel Sanábio Mazzetti Affonso

2026-08-29

Índice

Prefácio	3
1 Introdução	4
1.1 A importância do Python na Estatística	4
1.2 O escopo deste material	4
2 Configurando o Ambiente de Trabalho	6
2.1 O Ambiente <i>Python</i> e o pacote <i>reticulate</i>	6
2.2 Gerenciamento de Bibliotecas com o <i>pip</i>	6
2.3 Instalação Direta em Blocos de Código	6
3 Fundamentos	8
3.1 Filosofia, Sintaxe e Tipagem da Linguagem	8
3.2 Estruturas de Dados Nativas	9
3.3 Estruturas de Controle de Fluxo	9
3.4 Funções e Ferramentas Funcionais	10
3.5 Aplicação Prática: Manipulação de Listas	10
4 Referências	12

Prefácio

O presente material consiste na consolidação dos resultados de pesquisa obtidos durante o período de Iniciação Científica, no contexto do projeto **Métodos de Aprendizado Estatístico para Discriminação e Classificação: Aplicações à Indústria 4.0 – Fase 3**. Este documento visa não só o detalhamento dos métodos de coleta de dados via *Web Scraping*, mas também a viabilidade do uso do *Python* para tal, além de destacar as especificidades da linguagem e suas vantagens para o aprendizado de métodos estatísticos e computacionais.

Neste contexto, vale ressaltar que o material é de cunho introdutório, buscando viabilizar o aprendizado do *Python* enquanto linguagem de programação e do *Web Scraping* como técnica poderosa de coleta de dados. Dessa forma, seu foco é desenvolver, no estatístico, o interesse pelos diversos recursos computacionais que podem auxiliá-los enquanto profissionais e estudiosos da área..

A motivação para o desenvolvimento deste material surgiu da necessidade de diversificar as ferramentas de aplicação computacional existentes para analisar dados. A interseção entre a teoria estatística e a aplicação dos métodos se sustenta no casamento entre o rigor matemático e a modernidade computacional. Qualquer das partes que for negligenciada ocasionará problemas ou limitações importantes.

Ademais, é válido ressaltar a seção extra presente no material, desenvolvida com o intuito de explicar a estrutura dos livros digitais, como esse, usando o *Quarto*. Ainda que muitos considerem adequado o uso do *Bookdown* para este tipo de trabalho (o que é totalmente válido, dadas as especificidades do Rmarkdown), o *Quarto* se mostrou uma espécie de sucessor natural muito adequado, apresentando ferramentas modernas e extremamente úteis para detalhes de *layout*, especialmente tratando de documentos compilados em *HTML*.

Por fim, deixo aqui minha gratidão à Universidade Federal de Juiz de Fora (UFJF) pelo ambiente acadêmico. Agradeço, especialmente, ao meu orientador, Professor Doutor Lupércio França Bessegato, pela oportunidade e pelo acompanhamento técnico que possibilitaram o desenvolvimento deste trabalho.

1 Introdução

Na prática, a estatística contemporânea vai muito além do domínio teórico dos fundamentos, da modelagem probabilística e da inferência. A sociedade atual convive com uma realidade em que os dados são o produto mais valioso existente, além simplesmente de fontes de informação. Dessa forma, observa-se nos últimos anos o surgimento de conceitos como o *Big Data*, para descrever o fenômeno da quantidade massiva de dados que alteram significativamente o fluxo de trabalho do analista de dados, exigindo deste profissional, muitas vezes, muito além da capacidade de analisar dados. É necessário ter aptidão para manusear os dados apropriadamente, buscá-los em diferentes fontes, criar filtros específicos relativos às necessidades de um determinado problema, entre outras habilidades.

Neste contexto, este material foi elaborado com o intuito de introduzir a linguagem *Python* no arsenal analítico de estudantes e profissionais de Estatística, demonstrando sua viabilidade e potência para a análise de dados e a automação de rotinas de trabalho.

1.1 A importância do Python na Estatística

Tradicionalmente, a linguagem *R* ocupa um lugar importante na formação do estatístico, por ser uma linguagem vetorizada e já desenvolvida para o trabalho com dados. Porém, apesar do poder do *R* e seus pacotes nos trabalhos de modelagem de dados e no uso de testes clássicos (paramétricos e não-paramétricos), o *Python* consolidou-se como uma linguagem extremamente versátil, que permite aplicações robustas (como as de *Machine Learning*, por exemplo) e a integração de sistemas.

Em termos de aprendizado, o *Python* é facilmente assimilado por quem já teve contato com algoritmos, por conta de sua similaridade com linguagens compiladas, como no caso do *C++* (exigindo apenas adaptações sintáticas), e com linguagens interpretadas, como o *R*.

Na prática, o *Python* apresenta sua maior vantagem ao analisarmos sua quantidade de usuários e, consequentemente, sua quantidade de bibliotecas desenvolvidas para diferentes aplicações. No contexto da Estatística, destacamos o *pandas*, para estruturação de dados tabulares, e o *numpy*, para computação numérica de alta performance. O domínio dessas ferramentas permite ao analista processar grandes volumes de dados com eficiência, além de explorar diferentes possibilidades, como a de romper com a dependência de bancos de dados pré-estruturados através da automatização da coleta de dados via *internet*.

1.2 O escopo deste material

Este material tem um teor introdutório e prático. Ele não tem a intenção de esgotar as funcionalidades do *Python* (que seria, inclusive, uma tarefa praticamente impossível), tampouco de aprofundar-se nas teorias estatísticas que fundamentam nossas análises ou na teoria computacional

que fundamenta e explica as especificidades da linguagem. O objetivo central é ser uma ponte rápida para profissionais de Estatística compreenderem a linguagem e o ecossistema, e como podem se beneficiar deles.

Neste âmbito, a abordagem adotada se resume em execução de códigos e análise de resultados práticos, de forma que o conhecimento seja construído sequencialmente, como se segue:

- Capítulo 2 - Configurando o Ambiente de Trabalho: Apresenta as instruções necessárias para preparar o ambiente de desenvolvimento, abordando a integração do *Python* dentro do *RStudio* (via pacote *reticulate*) e o gerenciamento de bibliotecas.
- Capítulo 3 - Fundamentos do *Python* para Análise de Dados: Revisa as operações básicas, tipos de dados e a criação de funções, estabelecendo a base sintática da linguagem.
- Capítulo 4 - Manipulação e Análise Exploratória de Dados: Introduz a biblioteca *pandas* e a criação de visualizações, utilizando conjuntos de dados clássicos (como *Titanic* e *Iris*) para demonstrar o tratamento e a exploração de informações categóricas e numéricas.
- Capítulo 5 - Computação Numérica com *NumPy*: Foca no uso de arrays multidimensionais e operações vetorizadas, ilustrando seu poder computacional através de simulações estatísticas.
- Capítulo 6 - Coleta de Dados via *Web Scraping*: Demonstra como construir bases de dados próprias automatizando a extração de informações de páginas da internet, sempre pautado por boas práticas e ética na raspagem de dados.
- Capítulo 7 - Estrutura de Livros Digitais (via *Quarto*): Explica brevemente a estrutura de arquivos e códigos para o desenvolvimento de livros digitais, utilizando o *Rmarkdown* para os códigos e textos e a ferramenta *Quarto* para a compilação.

Espera-se que, ao final deste conteúdo, o estudante ou profissional de Estatística sinta-se confortável para importar, manipular e coletar dados em *Python*, além de estar apto a construir livros digitais que podem ser ferramentas ainda mais robustas que os relatórios padrão, tendo base sólida para estudos posteriores mais aprofundados nestas ferramentas e suas possibilidades.

2 Configurando o Ambiente de Trabalho

Antes de iniciarmos a manipulação de dados e as análises, é fundamental garantir que o ambiente computacional esteja configurado corretamente. Para a maioria dos estudantes e profissionais de Estatística, o *RStudio* é o ambiente de desenvolvimento integrado (IDE) mais familiar. A boa notícia é que o *RStudio*, especialmente quando aliado ao *Quarto*, oferece um suporte excelente para a execução de códigos em *Python*.

2.1 O Ambiente *Python* e o pacote *reticulate*

Em alguns casos, especialmente na primeira execução de um script *Python* dentro do *RStudio*, o ambiente pode não estar perfeitamente configurado. Isso ocorre porque o *RStudio* utiliza um pacote chamado *reticulate* para fazer a ponte e integrar o *Python* ao ecossistema do *R*.

O *reticulate* pode acabar criando, de forma automática, um ambiente virtual *Python* próprio, isolado daquele que você eventualmente já tenha instalado em seu sistema operacional. Por conta desse isolamento, esse novo ambiente nem sempre possui todas as bibliotecas necessárias para a análise de dados já pré-instaladas.

2.2 Gerenciamento de Bibliotecas com o *pip*

Caso apareçam mensagens de erro indicando que bibliotecas fundamentais como *pandas*, *jupyter* ou outras não foram encontradas, elas podem ser facilmente adicionadas utilizando o *pip*. O *pip* é o gerenciador de pacotes padrão do *Python*, responsável por baixar, instalar e atualizar bibliotecas externas.

Para realizar a instalação via **Terminal** (aba **Terminal** no *RStudio*, e não no Console), basta executar os seguintes comandos:

```
python -m pip install pandas numpy seaborn matplotlib
python -m pip install jupyter
```

2.3 Instalação Direta em Blocos de Código

Em documentos interativos (como notebooks *Jupyter* ou arquivos *.qmd* do *Quarto*), também é possível executar comandos de terminal diretamente de dentro das células de código (*code chunks*). Para isso, utiliza-se o caractere de exclamação (!) no início da linha.

Por exemplo, o comando a seguir instala a biblioteca *pandas* diretamente a partir do ambiente de execução do documento, sem a necessidade de abrir o terminal do sistema:

```
!python -m pip install pandas
```

Dica importante: Caso o *RStudio* continue apresentando problemas ou pareça estar utilizando uma instalação do *Python* diferente da esperada, pode ser necessário reiniciar a sessão do *R* (*Session > Restart R*) para que o *reticulate* reconheça as novas bibliotecas instaladas. Em geral, após essa configuração inicial e a instalação dos pacotes, o código passará a funcionar normalmente.

3 Fundamentos

Neste capítulo, iremos contextualizar a lógica de algoritmos ao *Python*, explicando brevemente a sua sintaxe essencial. A lógica de programação se mantém a mesma independentemente do ambiente computacional, mas detalhes estruturais como a tipagem de dados, a delimitação de blocos e a indexação variam significativamente de uma linguagem para a outra (Sebesta, 2018).

Enquanto linguagens tipadas estaticamente exigem a declaração prévia do tipo de cada variável (seja ela um número inteiro ou um texto), o *Python* utiliza a tipagem dinâmica, inferindo o tipo do dado no momento de sua execução (Lutz, 2013). Além disso, diferentemente de linguagens que utilizam chaves para agrupar comandos, o *Python* adota a indentação obrigatória como regra sintática, o que impõe a elaboração de códigos padronizados e altamente legíveis. Outro fator de atenção para os analistas e desenvolvedores é a indexação: ao contrário do *R* (que é também uma linguagem amplamente utilizada no contexto de análise de dados e Estatística), onde o primeiro elemento de uma estrutura de dados ocupa o índice 1, o *Python* adota a indexação baseada em zero (0-based), característica compartilhada com linguagens como *C* e o *C++*.

3.1 Filosofia, Sintaxe e Tipagem da Linguagem

Grande parte das linguagens de programação surgem (ou evoluem) focadas em otimização de performance para a máquina. Porém, esse não é o caso do *Python*, que foi concebido a partir de uma filosofia muito bem fundamentada e orientada ao desenvolvedor. O princípio norteador da linguagem é o de que um código é lido muito mais vezes do que ele é escrito. Portanto, a clareza e a legibilidade devem ser prioridades absolutas.

Essa filosofia está muito bem documentada no Zen do *Python* (PEP 20), um conjunto de princípios que guiam as melhores práticas na linguagem, destacando máximas como “o simples é melhor que o complexo” e “a legibilidade conta” (Peters, 2004).

Para garantir essa legibilidade de forma estrutural, a sintaxe do *Python* elimina o uso de marcadores visuais excessivos, como as chaves `{}` ou as expressões `begin/end` para delimitar blocos de código. Em seu lugar, a linguagem adota a indentação obrigatória. O recuo da margem esquerda não é meramente uma questão de estilo, mas uma regra sintática rígida documentada na PEP 8 (Van Rossum; Warsaw; Coghlan, 2001). Seu objetivo é forçar o programador a organizar, hierarquicamente, as operações, resultando em um código mais limpo.

Além da sintaxe visual, o *Python* caracteriza-se por uma tipagem dinâmica e forte. Dinâmica porque não exige a declaração prévia do tipo da variável - a própria linguagem infere se o dado é numérico ou textual no momento da atribuição, além de permitir mudanças de valor e de tipo durante o processo de desenvolvimento. Forte porque, uma vez inferido o tipo, o *Python* não realiza conversões implícitas que possam gerar comportamentos inesperados, garantindo a segurança e a consistência das operações.

3.2 Estruturas de Dados Nativas

Conhecer as estruturas de dados nativas do *Python* é essencial para armazenar e organizar informações. As principais coleções embutidas na linguagem são as listas (`list`), as tuplas (`tuple`), os dicionários (`dict`) e os conjuntos (`set`).

No contexto da manipulação de dados e da programação geral, a **lista** destaca-se como a estrutura mais versátil e frequentemente utilizada. Diferentemente dos vetores atômicos encontrados no *R* ou dos *arrays* estáticos comuns no *C++*, as listas em *Python* são coleções heterogêneas e dinâmicas. Isso significa que uma única lista pode armazenar, de forma simultânea, números inteiros, valores reais, textos ou até mesmo outras listas em seu interior (Lutz, 2013).

Além da heterogeneidade, as listas são estruturas **mutáveis**. Após sua criação, o programador ou analista pode alterar seu conteúdo em tempo de execução sem a necessidade de realocar a estrutura inteira na memória. É possível modificar valores existentes através de seus índices, adicionar novos elementos ao final da coleção (utilizando o método `append()`) ou remover o último item inserido (através do método `pop()`) (Python Software Foundation, 2026a). Essa flexibilidade de manipulação direta torna as listas uma base poderosa para o tratamento inicial de dados antes de avançarmos para o uso de bibliotecas.

3.3 Estruturas de Controle de Fluxo

A espinha dorsal de qualquer linguagem de programação é a capacidade de direcionar a execução dos algoritmos, através de estruturas de controle de fluxo. Destacam-se, nesse contexto, as condições lógicas e os laços de repetição, que são amplamente conhecidos e utilizados por quem trabalha com códigos, seja qual for o contexto. Em *Python*, as estruturas de controle de fluxo seguem a filosofia da linguagem - citada neste capítulo - sendo projetadas para serem diretas e facilmente legíveis, se comparadas a outras linguagens de programação.

As estruturas condicionais utilizam as palavras-chave `if`, `elif` (uma abreviação de `else if`) e `else`. A condição avaliada não exige o uso de parênteses, e o bloco de código subordinado é demarcado de forma simples e intuitiva: através da inserção de dois pontos (`:`) ao final da instrução, seguida da já mencionada indentação obrigatória.

No que diz respeito às estruturas de repetição, o *Python* oferece os laços `while` e `for`. O laço `while` opera de maneira tradicional, executando um bloco de código enquanto uma condição permanecer verdadeira. Contudo, o laço `for` apresenta uma diferença fundamental e vantajosa em relação a linguagens compiladas clássicas. Em vez de iterar primariamente sobre uma progressão aritmética de índices, o `for` em *Python* atua como um iterador direto sobre os itens de qualquer sequência, como uma lista ou uma tupla (Python Software Foundation, 2026a).

No âmbito da análise de dados, por exemplo, essa característica permite que o analista percorra os dados de uma coleção diretamente (por exemplo, lendo cada observação de uma variável) sem a necessidade de gerenciar variáveis contadoras, o que reduz significativamente a chance de erros lógicos. Caso uma iteração numérica tradicional seja estritamente necessária, a linguagem fornece uma função embutida `range()`, capaz de gerar sequências numéricas de forma eficiente. É importante destacar que a função `range()` não armazena a sequência inteira na memória de uma só vez; ela calcula os valores sob demanda a cada passo da iteração. Consequentemente, sua complexidade de

espaço (consumo de memória) é constante, $O(1)$ ¹, independentemente de a sequência possuir dez ou dez milhões de elementos (Python Software Foundation, 2026b). Isso evita gargalos de performance e torna o código otimizado.

3.4 Funções e Ferramentas Funcionais

O processo de desenvolvimento de um algoritmo não apresenta **obrigatoriedade** de práticas que vão além de uma lógica concisa e do respeito à sintaxe da linguagem. Porém, a escalabilidade de um *software* ou a estrutura de uma análise de dados, por exemplo, exigem boas práticas que são fundamentadas no bom uso das funções (métodos), antes de partir para estruturas mais complexas. No *Python*, a definição de uma função é realizada através da palavra-chave **def**, seguida do nome da função e de seus respectivos parâmetros. Assim como nas funções prontas do *Python*, as funções customizadas também exigem, por padrão da sintaxe, os parênteses logo após o nome da função. Portanto, os parâmetros são criados dentro dos parênteses, e o valor de saída da operação, ao final da função, é especificado pelo comando **return**.

Contudo, como citado neste capítulo, o *Python* não se limita ao paradigma procedural; ele incorpora elementos poderosos do paradigma de programação funcional. Para evitar a escrita de laços **for** muito extensos ao transformar bases de dados, a linguagem fornece funções de alta ordem (*higher-order functions*), sendo as mais notáveis o **map()** e o **filter()** (Python Software Foundation, 2026c).

Para aqueles que estão habituados ao uso da linguagem *R*, muito usada na Estatística e na análise de dados, essa mecânica é conceitualmente idêntica ao uso de funções **apply()** (como o **lapply()** e o **sapply()**). O objetivo é o mesmo: permitir um processamento de dados mais rápido, conciso e com menos linhas de código. O **map()** aplica uma mesma transformação a todos os itens de uma coleção de forma simultânea, enquanto o **filter()** extrai e retorna apenas os elementos que satisfazem uma determinada condição lógica (bem próximo do que faz a função **which()**, no *R*).

Essas ferramentas são ainda mais ágeis quando combinadas com as expressões **lambda**. Uma função **lambda** é uma função anônima — ou seja, uma função que não precisa ser formalmente nomeada com **def** e que é criada em uma única linha de código, sendo ideal para operações rápidas ou descartáveis que serão usadas apenas uma vez dentro de um **map** ou **filter** (Lutz, 2013).

3.5 Aplicação Prática: Manipulação de Listas

No script a seguir, consolidamos todos os fundamentos apresentados neste capítulo. O exemplo ilustra desde a manipulação direta de uma lista utilizando métodos nativos (como o **append()** e o **pop()**), até o uso avançado das funções **map()** e **filter()** associadas a funções anônimas (**lambda**). Por fim, demonstramos a aplicação de uma função customizada, desenvolvida com **def()**, para o cálculo de uma média simples.

¹A notação $O(1)$ faz parte da chamada “Notação *Big O*” (*Big O Notation*), utilizada na Ciência da Computação para descrever matematicamente a eficiência e a escalabilidade de um algoritmo. Uma complexidade de espaço $O(1)$, ou constante, indica que a quantidade de memória exigida para a execução da tarefa permanece exatamente a mesma, não importando se a entrada possui dez ou dez milhões de elementos.

Nota-se, também, o uso das chamadas *f-strings* (`f"texto {variável}"`) dentro da função `print()`. Esse é o método mais moderno e recomendado no *Python* para interpretar textos e variáveis na mesma linha, gerando saídas de telas mais limpas.

```
# ---- Definição de uma função customizada ----
def media(lista):
    soma = sum(lista)
    quantidade = len(lista)
    return soma / quantidade

# ---- Criação e manipulação direta da lista ----
valores = [10, 20, 30, 40]
print(f"Lista original: {valores}")

# Uso do append()
valores.append(50)
print(f"Após append (adicionar 50): {valores}")

# Uso do pop()
valores.pop()
print(f"Após pop (remover último elemento): {valores}")

# ---- Uso de ferramentas funcionais (map, filter e lambda) ----

# Uso do map() com lambda
valores_dobrados = list(map(lambda x: x * 2, valores))
print(f"Valores dobrados com map: {valores_dobrados}")

# Uso do filter() com lambda
valores_filtrados = list(filter(lambda x: x > 20, valores))
print(f"Valores > 20 com filter: {valores_filtrados}")

# ---- Aplicação da função customizada ----
resultado = media(valores)
print(f"Média final dos valores originais: {resultado}")
```

```
Lista original: [10, 20, 30, 40]
Após append (adicionar 50): [10, 20, 30, 40, 50]
Após pop (remover último elemento): [10, 20, 30, 40]
Valores dobrados com map: [20, 40, 60, 80]
Valores > 20 com filter: [30, 40]
Média final dos valores originais: 25.0
```

4 Referências

LUTZ, Mark. **Learning Python: Powerful Object-Oriented Programming**. 5. ed. *[S.l.]*: O'Reilly Media, Inc., 2013.

PETERS, Tim. **PEP 20: The Zen of Python**. *[S.l.]*: Python Software Foundation, 2004. Disponível em: <<https://peps.python.org/pep-0020/>>.

PYTHON SOFTWARE FOUNDATION. **The Python Tutorial: Data Structures**. *[S.l.: S.n.]*.

PYTHON SOFTWARE FOUNDATION. **Built-in Types - Ranges**. *[S.l.: S.n.]*.

PYTHON SOFTWARE FOUNDATION. **Built-in Functions**. *[S.l.: S.n.]*.

SEBESTA, Robert W. **Conceitos de Linguagens de Programação**. 11. ed. *[S.l.]*: Bookman Editora, 2018.

VAN ROSSUM, Guido; WARSAW, Barry; COGHLAN, Nick. **PEP 8: Style Guide for Python Code**. *[S.l.]*: Python Software Foundation, 2001. Disponível em: <<https://peps.python.org/pep-0008/>>.